SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO5: *Apply computer vision techniques to real-world applications such as face recognition, tracking, medical imaging, and autonomous systems. (BL3)*

Assessment Tool Weightage: 30%

QUESTIONS: 2 Total Marks:30 Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I CH.V.Ramayogi/192424319(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: CH.V.Ramayogi

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	

Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. Scenario: Design an algorithm to detect faces in an image and recognize them using the Eigenfaces method.

Task:

A. Input: A grayscale image (size M × N)
B. Output: Bounding boxes of detected faces and corresponding recognized names
C. Requirements: Handle multiple faces in the same image and varying lighting conditions

Pseudocode Task:

1. Preprocess the image (grayscale normalization, histogram equalization)
2. Detect faces using a sliding window or Viola-Jones detector 3. Extract features using Eigenfaces
4. Compare with database of known faces using Euclidean distance
5. Output bounding boxes with recognized labels

2. Scenario: Design an algorithm to detect road lanes in a real-time driving video sequence.

Task:

A. Input: Video frames from a front-facing camera
B. Output: Lane lines highlighted on each frame
C. Requirements: Robust to shadows, curves, and partial occlusion

Pseudocode Task:

1. Capture video frame
2. Convert frame to grayscale
3. Apply Gaussian smoothing to reduce noise
4. Perform edge detection (Canny)
5. Apply Hough Transform to detect straight/curved lines
6. Overlay detected lanes on the original frame
7. Repeat for all frames

1. Face Detection and Recognition Using Eigenfaces

Aim

To design an algorithm for detecting multiple human faces in a grayscale image and recognizing them using the **Eigenfaces method**, while handling different lighting conditions.

Input

A grayscale image of size **M × N** containing one or more faces.

Output

- Bounding boxes around detected faces
- Recognized person's name for each face
- **Unknown** label if no matching face is found

Algorithm / Pseudocode

START

INPUT grayscale image I of size M × N

1. Read the input grayscale image.
2. Normalize the pixel intensity values to reduce
   differences caused by illumination.
3. Apply histogram equalization to improve contrast
   and handle varying lighting conditions.
4. Initialize the Viola-Jones face detector.
5. Detect all face regions present in the image.
6. IF no faces are detected THEN
   Display "No Face Detected"
   STOP
   END IF
7. FOR each detected face region DO
   a. Extract the detected face region.
   b. Resize the face into a standard size.
   c. Normalize the extracted face image.
   d. Convert the face image into Eigenface
   feature representation.

e. Project the face onto the Eigenface space.

f. Compare the extracted feature vector with
all known face feature vectors in the database.

g. Calculate Euclidean distance for each comparison.

h. Find the minimum Euclidean distance.

i. IF minimum distance is less than the
predefined recognition threshold THEN
Assign the corresponding person's name.
ELSE
Assign the label "Unknown".
END IF

j. Draw a bounding box around the detected face.

k. Display the recognized name near the
bounding box.

END FOR

8. Display the final image containing all detected
faces and their recognized names.

OUTPUT bounding boxes and recognized labels

STOP

Result

The algorithm detects multiple faces in the input image and recognizes each face using **Eigenface features and Euclidean distance matching**.

2. Real-Time Road Lane Detection

Aim

To design an algorithm for detecting and highlighting road lanes in real-time video captured from a front-facing camera.

Input

Video frames captured continuously from a vehicle's front-facing camera.

Output

Each video frame with detected road lanes highlighted.

Algorithm / Pseudocode

START

1. Open the input video or front-facing camera.

2. Initialize video frame capture.

3. WHILE video frames are available DO

    a. Capture the current video frame.

    b. Resize the frame if required for faster
    real-time processing.

    c. Convert the color frame into a grayscale image.

    d. Apply Gaussian smoothing to reduce noise
    and small unwanted intensity variations.

    e. Apply brightness normalization to reduce
    the effect of shadows and lighting changes.

    f. Define the Region of Interest (ROI)
    containing only the road area.

    g. Apply Canny edge detection to identify
    possible lane boundaries.

    h. Apply the Hough Transform to detect
    straight or curved lane line segments.

i. Remove unwanted lines based on their slope, position, and orientation.

j. Separate the left lane and right lane.

k. Use information from previous frames to improve detection during partial occlusion.

l. Estimate curved lanes using multiple detected lane points when required.

m. Draw the final detected lane lines.

n. Overlay the highlighted lane lines on the original video frame.

o. Display the processed output frame.

END WHILE

4. Release the camera or video resource.

5. Close all display windows.

OUTPUT real-time video with highlighted lane lines
STOP

Result

The algorithm processes each video frame and detects road lane boundaries using **Gaussian smoothing, Canny edge detection, Region of Interest selection, and Hough Transform**, then overlays the detected lanes on the original frame.